

FLEET LAMBDA PLATFORM

Real-time ride-hailing operations and daily profitability reconciliation

EC8203 Applied Big Data Engineering Mini-Project

Author	Programme
Udara Subodhitha Senevirathna	BSc Computer Engineering
Institution	University of Ruhuna
Report date	22 September 2026

Submission statement. This report describes the delivered implementation, measured checks and deliberate deferrals. It does not claim a recorded demo video, production-scale throughput, Grafana/MinIO deployment or formal event-time windows.

1. Use case and requirements

A ride-hailing operator needs immediate visibility into fleet utilization and earnings while reconciling completed trips against daily fuel and maintenance costs. The two views have different truth requirements: live state may be indicative, while daily profit must not treat missing telemetry or missing costs as zero.

Business questions

- What is current fleet utilization, idle ratio, trip activity and earnings by zone and time of day?
- Which vehicles are genuinely unprofitable after yesterday's costs, and which results remain unknown because evidence is incomplete?
- Can operators detect stalled ingestion, failed reconciliation, missing dates, damaged exports and bad source rows?

Interpreted requirements

ID	Requirement	Acceptance evidence
R1	Continuous Python telemetry through Kafka partitions	Checkpointed Spark query and fresh-install smoke test
R2	One expense file per simulated day	Producer creates dated CSV; Airflow discovers closed days
R3	Meaningful cleaning, enrichment, aggregation and join	Native validation, time bucket, trip aggregate, coverage and cost join
R4	Queryable serving and consolidated result	PostgreSQL, FastAPI, JSON report and business results page
R5	Observable health and failures	Structured JSON logs; ingestion and report-integrity HTTP rules
R6	Reproducible local execution	Compose, migrations, unit/integration tests and empty-volume verification

The persistent simulation clock starts at 2026-03-01 00:00 UTC. One simulated day equals 300 real seconds. A telemetry tick occurs every two real seconds and advances 9.6 simulated minutes. Times are UTC and monetary values are stored as integer LKR cents.

2. Architecture decision: Lambda vs Kappa

Lambda was selected because corrected daily cost files must restate historical accounts while immutable trip evidence remains replayable. The speed path gives operational visibility; the batch path recomputes a vehicle-day result from committed archives and the latest valid expense file.

Decision factor	Lambda in this project	Kappa alternative
Latency	Live state and lookbacks update every micro-batch.	A single stream could also meet low latency.
Replay	Spark batch rereads verified Parquet by day.	Both expenses and telemetry must be durable versioned events.
Consistency	Daily output is restated from a fixed input snapshot and marked by run/version.	Historical keyed state needs correction/retraction semantics.
Cost/complexity	Two code paths but simple local recovery and explainable accounting.	One topology in principle, but more state/replay operational complexity here.
Failure isolation	Live metrics can continue when one expense date fails.	A unified topology can couple operational and accounting failures.

Rejected alternative

Kappa was rejected for this two-week local platform, not because Kappa is inferior in general. It becomes attractive when all sources are naturally versioned event streams and the team can operate replay, state migration and retractions. Daily CSV corrections make the separate batch truth path easier to verify and defend.

Trade-off

Lambda duplicates identity and transformation logic. This risk is mitigated by a shared fixed event-field contract, UTC timestamp canonicalization, parity tests, persistent identities, deterministic daily recomputation and explicit completeness states. Eventual consistency remains: a late correction is visible after the next Airflow restatement.

Consistency promise: no end-to-end exactly-once claim. Kafka checkpoints, event identities, trip keys, archive manifests, database transactions and idempotent restatement provide practical replay safety within the documented single-writer dataset contract.

3. Delivered architecture and data flow

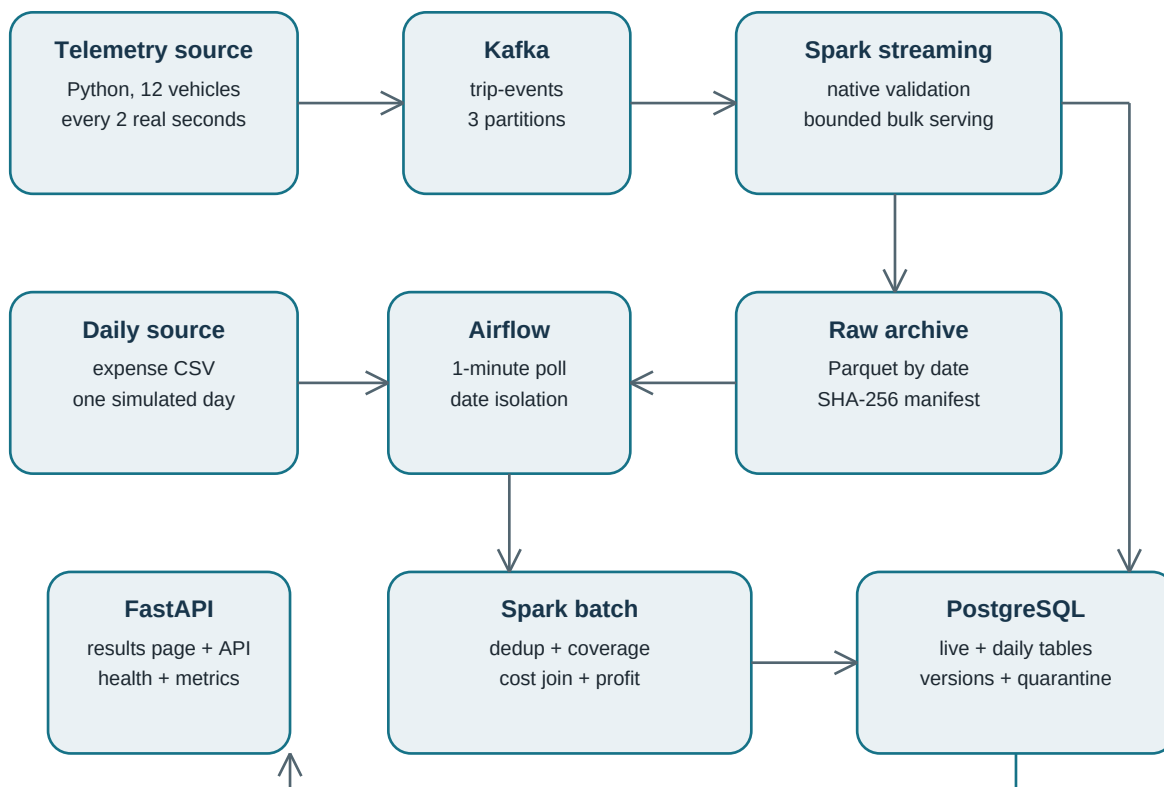


Figure 1. Delivered local Lambda architecture. Arrows show the primary data flow; PostgreSQL serves FastAPI. The raw archive and database/checkpoint are operated as one dataset.

The single Spark streaming query validates Kafka records, archives valid events and applies bounded bulk serving. A session advisory lock fences archive writes; a transaction lock protects the commit ledger. Airflow invokes Spark batch reconciliation independently for each ready date, newest first, with a configurable five-changed-date budget.

Storage contracts

- Raw Parquet files are accepted only through committed batch manifests containing row counts and SHA-256 digests.
- PostgreSQL stores latest vehicle state, completed trips, persistent event identities, conflicts, quarantine, run history and daily profitability.
- Daily JSON contains run ID, algorithm version, quality coverage and vehicle results. Its SHA-256 is stored in PostgreSQL and checked by health/retry logic.

4. Technology selection

Component	Why selected	Constraint / rejected alternative
Kafka 3.9	Required ingestion technology; keyed events and three partitions demonstrate partitioned transport.	Single broker is suitable only for a laptop; no replication.
Spark 3.5 Structured Streaming	Required processing choice; native schema validation and checkpointed Kafka consumption.	Storm rejected to keep one engine across stream and batch.
Spark DataFrames (batch)	Distributed trip aggregation, conflict detection, coverage and expense join.	Pure Python dictionaries were removed from production batch processing.
Airflow 2.10	Scheduled discovery, retries, UI and serialized daily reconciliation.	One visible task is simpler but offers less stage-level retry detail.
PostgreSQL 16	Transactions, constraints, JSON metadata and queryable serving.	Cassandra is less suited to relational reconciliation and local transactions.
Parquet	Columnar immutable replay source with typed timestamps and date partitions.	MinIO/S3 adds deployment realism but not required laptop value.
FastAPI	Typed validation, API documentation, health endpoints and a small business view.	Airflow UI is not a fleet dashboard.
Docker Compose	Repeatable single-host wiring and persistent named volumes.	Not production orchestration.

The implementation pins major Python dependencies and container images. Host tests use Python 3.11 where possible; CI installs the development requirements and validates Compose configuration. Local demo credentials and loopback-only ports are not suitable for external deployment.

5. Implementation

Streaming path

- Python emits telemetry for twelve vehicles; every fourth vehicle remains parked to create an interpretable idle/cost-only case.
- Spark uses from_json plus native column constraints. Unknown vehicles, malformed JSON, impossible coordinates/speed, invalid types and out-of-bounds simulated timestamps are rejected.
- Micro-batches are capped at 2,500 rows and Kafka at 2,000 offsets per trigger. Accepted rows are collected only inside this explicit classroom bound, then written with bulk SQL operations.
- Persistent event fingerprints deduplicate across batches. Same identity with different business content becomes a conflict; implicated trip revenue and live state are removed rather than arbitrarily selected.

Batch path

- Only ledger-committed and checksum-verified Parquet enters reconciliation. Spark detects event/trip conflicts, aggregates completions by vehicle and calculates telemetry coverage.
- All registered vehicles remain visible. Complete telemetry plus expenses yields profit; missing telemetry, conflicts or missing expenses yields a null profit with a quality status.
- A greater-than-five-percent expense rejection rate fails that date without replacing the last good result. Other dates continue and the Airflow task reports a failure summary.
- Database rows move to pending before file export. Atomic file replacement then publishes a digest. Missing/corrupt published files are automatically regenerated.

Serving semantics

Live fleet and zone endpoints use latest-state and rolling simulated-time lookbacks. They are explicitly labelled indicative. The daily endpoint returns vehicle rows and publication metadata from one SQL statement, avoiding mismatched versions during concurrent publication. The browser page refreshes every 15 seconds and presents LKR values, report quality and algorithm version.

6. Observability and failure behaviour

Signal	Detection	Response / meaning
Live freshness	Accepted-event arrival age plus simulated event lag	HTTP 503 after 120 real seconds; duplicates cannot refresh health
Batch failures	Latest run status per date and stalled-run threshold	HTTP 503; failed date retained with error
Date completeness	Every expected date from simulation start to latest closed day	Missing historical hole makes report health degraded
Publication	Pending state, algorithm version and JSON SHA-256	Missing/corrupt/outdated report makes health degraded and triggers restatement
Data quality	Quarantine rows and >5% daily gate	Bad date fails; last good daily result remains
Tracing	trace_id in source/live trip records and structured logs	Supports event walkthrough; not full distributed tracing
Business alert	Observed idle_since against simulated latest event	API lists considerably idle vehicles

Structured JSON logs include timestamp, stage and message plus batch/run identifiers, row counts and errors where relevant. The /metrics endpoint exports fixed numeric Prometheus text, but no Prometheus server, notification channel or Grafana instance is claimed.

Failure exercise

Final verification stopped only the telemetry producer. Ingestion health returned HTTP 503 at a 126-second event age, then the cleanup handler restarted the producer and health recovered to HTTP 200 at 5.6 seconds. The demo runbook repeats this exercise. Because the simulation wall clock continues, an outage creates an honest historical coverage gap; the system must not fabricate events or profit.

Recovery rule: PostgreSQL, Kafka, Parquet and Spark checkpoint form one logical dataset. Coordinated backups are required. Deleting one component alone is unsupported. Conflicts require audited source correction and a deliberate rebuild, not ad-hoc deletion.

7. Results and business output

The running platform exposes a consolidated business page at <http://localhost:8001>. It combines current reporting/active vehicles, idle ratio, hourly earnings, zone activity and a selectable daily vehicle profitability table. It also states the report run, publication state, algorithm version and quality outcome.

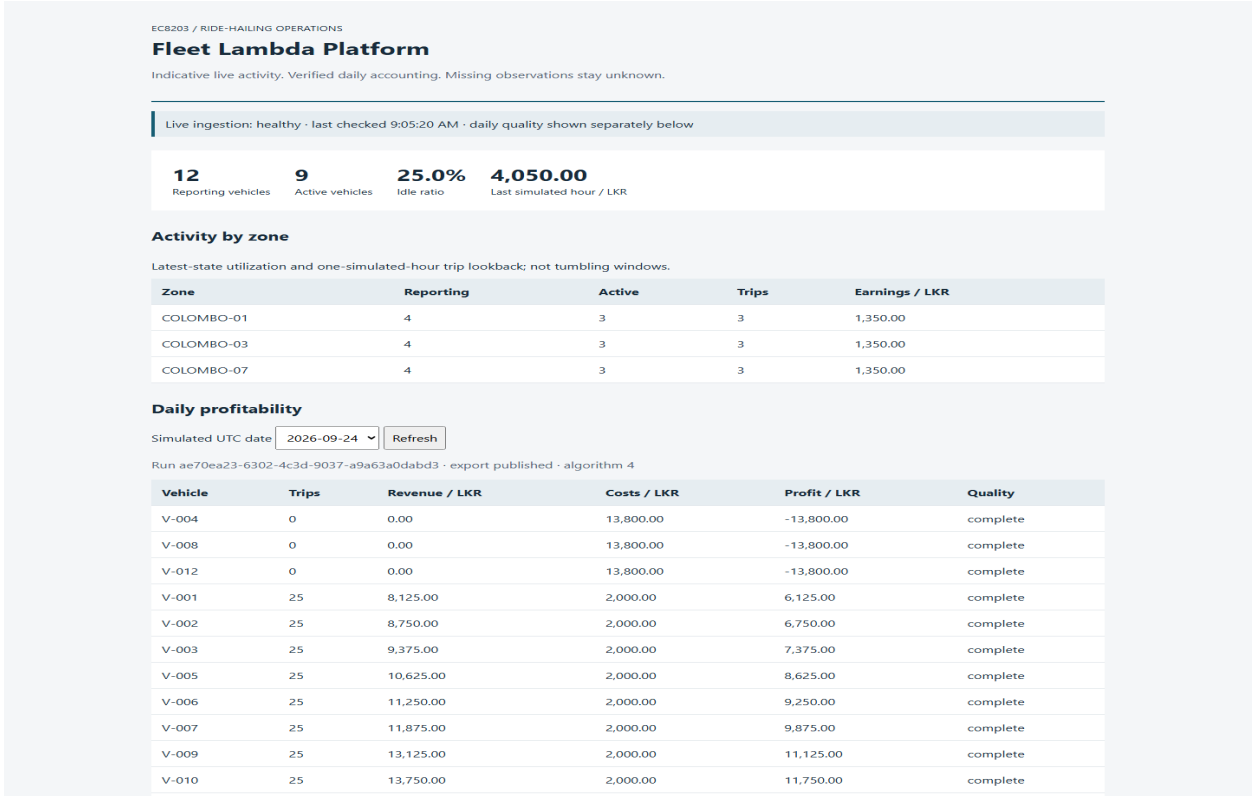


Figure 2. Actual local results page captured after the version-4 restatement. The visible table shows genuine parked-vehicle losses and completed profitable vehicle-days.

Verified snapshots

Evidence	Observed result
Automated unit/API/archive tests	39 passed after completion fixes
Isolated Spark/PostgreSQL suite	23 named checks passed; disposable schema and temporary files
Fresh volumes	Eight long-running services healthy/running; DAG imports clean; report and live smoke checks passed
Final report-integrity health	Healthy; 209 dates; zero missing, invalid, outdated, failed, stalled or pending
Final result quality snapshot	788 complete vehicle-days; 1,720 incomplete-telemetry vehicle-days

The historical counts are a dated snapshot, not a performance benchmark. Incomplete days reflect real downtime in the persistent demonstration dataset. All retained dates were restated under algorithm version 4 so export digests and identity semantics were recalculated rather than inherited.

8. Verification and reproducibility

The verification strategy separates pure logic, API behaviour, isolated database/Spark integration, live smoke checks and a fresh-volume installation. Faults that would damage evidence are applied only to temporary directories and UUID-named schemas.

Area	Checks
Replay/integrity	Cross-batch duplicate, timestamp spelling parity, conflicting fare, committed archive loss, writer fence
Financial truth	Trip counts, zero-revenue parked vehicle, corrected cost, missing expense, incomplete telemetry, profit equation
Publication	Pending state, simulated replace failure, retry, missing JSON recovery, corrupt JSON recovery, digest/run match
Validation	Malformed JSON, unknown vehicle, future timestamp and daily cost quality gate
Concurrency	Same-date Airflow lock and consistent one-statement API read
Operations	Compose schema, DAG imports, all endpoints, empty PostgreSQL/Kafka/archive/checkpoint volumes

Fresh-install evidence

An isolated Compose project used empty uniquely named volumes, alternate localhost ports and the already built images. It generated live data and a daily report, passed the read-only smoke test, showed no Airflow import errors, and was then stopped. Its volumes were retained for inspection. This validates initialization on the same Docker host; it does not claim an uncached second-machine download test.

Reproduction

- Copy `.env.example` to `.env` only when no real `.env` exists, then run `docker compose up --build -d`.
- Run `python -m unittest discover -s tests -v` and `python -m scripts.verify_running --wait-seconds 480`.
- Run the isolated reconciliation verification in the application image; it cleans only its UUID schema.
- Use `scripts/verify_clean_install.py` when alternate ports 18001/18080 are free; it preserves test volumes and stops containers.

9. Limitations, scale and security

Current limitation	Production direction
Single Kafka broker / local[2] Spark	Replicated Kafka, distributed Spark and capacity testing
Bounded driver collection and PostgreSQL merge	Partition staging and set-based final merge after measured load
Small immutable Parquet files	Object storage plus Iceberg/Delta compaction and retention
One Airflow task	Stage-specific tasks/sensors after operational need is demonstrated
SQL live lookbacks; no watermark finalization	Revise simulator cadence, then add event-time windows and explicit lateness policy
Local credentials, no API auth/TLS	Secrets manager, least-privilege roles, authentication, TLS and network policy
No Prometheus/Grafana notification stack	Scraping, dashboards, SLOs and routed alerts
Manual conflict resolution	Audited correction/rebuild workflow and lineage tooling
No measured 500 eps claim	Representative load generator, p50/p95, resource and database saturation tests

The project deliberately defers MinIO, separate consumers, formal windows, Grafana and durable alert history. The brief requires meaningful processing, a consolidated report or dashboard and at least one alert/health rule; adding optional infrastructure would not replace correctness, architecture reasoning or reproducible evidence.

Security scope

Host ports bind to loopback and credentials are documented as local-demo only. Before external deployment, rotate secrets, give FastAPI a read-only database role, separate migration/write roles, authenticate endpoints, enable TLS and define retention/privacy controls for location traces.

Ethical interpretation

Profitability is an operational vehicle measure, not a driver-performance score. Missing data and conflicts are shown as unknown to reduce harmful inference. A production system would require governance for location access, retention and human review of alerts.

10. Conclusion and references

The delivered platform meets the core mini-project objective: two simulated sources feed an observable Lambda pipeline; Spark supports both near-real-time processing and historical reconciliation; Airflow orchestrates daily work; results land in queryable PostgreSQL and a versioned consolidated report; and FastAPI exposes operational and financial answers.

The strongest engineering decision is not a particular tool but the treatment of uncertainty. Duplicate identities are idempotent, conflicts invalidate authority, missing telemetry does not become zero revenue, bad cost files preserve the last good result, and published files are checked rather than assumed. The result is a defensible teaching system with explicit boundaries.

References

- EC8203 Data Engineering Mini-Project brief, 2026, pages 1-5.
- Apache Kafka 3.9 documentation: <https://kafka.apache.org/39/>
- Apache Spark 3.5.6 Structured Streaming programming guide and Kafka integration documentation: <https://spark.apache.org/docs/3.5.6/>
- Apache Airflow 2.10.5 documentation: <https://airflow.apache.org/docs/apache-airflow/2.10.5/>
- PostgreSQL 16 documentation: <https://www.postgresql.org/docs/16/>
- FastAPI documentation: <https://fastapi.tiangolo.com/>

Submission artefacts

Artefact	Location / purpose
Source repository/ZIP	Complete code, Compose, migrations, tests and CI workflow
This PDF	Architecture decision, design, evidence, limitations and references
docs/DEMO_RUNBOOK.md	Prepared approximately eight-minute live demonstration
docs/FINAL_SCOPE.md	Authoritative delivered scope and deferrals
output/evidence/clean-install.json	Machine-readable fresh-volume verification evidence

Contribution note. The README identifies one author. No additional group-member contributions are invented. If submitted as group work, the students must add a truthful statement based on actual contributions.